themselves approved (you can request approval of the third-party platform via the processes described in this section). These platforms need to be approved themselves because they are not subject to our Anthropic agreement and so do not have the legal protections from that agreement.
 Tools that violate our GitLab Duo dogfooding principle such as (for example: Cursor and Cline). Team members may be permitted to use Cursor and Cline for competitive research purposes provided they review and follow our guidelines for competitive use.
 Anthropic API keys for any use-case not already approved.

SECTION: tools-and-tips/ai/claude.md

Learn how to use Claude.ai to infuse AI into your workflows, tools, and processes to greater efficiencies. You can use Claude to write content, create outlines and documents, summarize content, generate database migrations and SQL statements, recommend refactorings, and more.

 Access

Open claude.ai and use your team member email address for SSO login. You can also use the Claude tile in Okta. Review the usage guidelines and FAQs (internal).

 Resources

1. AI at GitLab initiative (internal)
   - Review the AI At GitLab usage guidelines and FAQ
1. Claude.ai support articles collection
1. How up-to-date is Claude's training data?
1. Join the ai-at-gitlab Slack channel

 Tips

> Note Only document public use cases and tips with Claude.ai, and keep everything else SAFE in the internal handbook.

Claude.ai can answer many different questions and topics. Be creative, curious, and explore, and iterate on the best chat prompts. Since GitLab Duo Chat also uses Anthropic Claude as one of the LLMs, you can test and repurpose similar chat prompts.

 Applications and CLI

1. Download Claude for Desktop to use the Claude application on macOS.
1. Anthropic API access
   - An Anthropic API key is required. Follow the guidance in the internal handbook.
1. Anthropic CLI (requires API access)
   - Learn about the Anthropic SDK and community projects for CLIs: anthropic-cli

 Token Limits

> [!note]

> This section covers tips for token limits in context windows, for API rate limits please review Anthropics rate limit documentation.

To help prevent hitting rate limits while using Claude, we recommend following the tips below:

- Set the appropriate thinking mode, Extended seems to support more context that you upload and generated content.
- Set the response mode to Concise to shorten Claudes response length.
- Before re-prompting, try editing your previous prompt to include your new ask.
- Be specific in your prompt, 3.7 likes to try and build out more than you ask, clarify to only make the changes requested in a more iterative approach.

> [!note]
> At the time of this writing, Claude 3.5 Sonnet, has higher rate limits, so if this does the job and you need long conversations, this may be a viable choice until rate limits on Claude 3.7 Sonnet are increased, although will be missing out on improvements made in the 3.7 version of the model.

Example Prompt Library

Below is a collection of useful prompts organized by division, with example use cases for each prompt to help team members leverage AI assistants effectively.

Sales Division

Value Proposition Analysis Prompt

markdown
```
Analyze how a company's features address key challenges in the [MARKET SEGMENT] space. Focus on:
1. Pain points solved
2. Feature advantages
3. Customer support benefits
4. Integration capabilities
5. ROI potential
```

Example Use Case: When preparing for a sales call with a prospect in the financial services sector.

Sales Email Template Generator

markdown
```
Generate a personalized sales email to [PROSPECT TYPE] who is currently using [CURRENT SOLUTION].
Include:
- Pain points they might be experiencing
- Specific features that address these pain points
- A clear value proposition
- Soft call-to-action for a demo
```

Keep the tone professional but conversational and limit to 200 words.

Example Use Case: Creating tailored outreach emails to development team leads at healthcare companies who are using fragmented tools and could benefit from a single application approach.

Marketing Division

Content Brief Creator

markdown
Create a detailed content brief for a [CONTENT TYPE] about [TOPIC] targeted at [AUDIENCE].
Include:
- Proposed title options (3-5)
- Key messages to convey
- SEO keywords to target
- Outline with section headings
- Suggested data points or case studies to include
- Call-to-action recommendations

Example Use Case: Planning a comprehensive blog post.

Social Media Campaign Planner

markdown
Develop a 2-week social media campaign promoting [FEATURE/PRODUCT] across LinkedIn and Twitter.
For each platform, create:
- 5 unique post ideas with copy variants (280 chars for Twitter, 700 chars for LinkedIn)
- Hashtag recommendations
- Suggested posting schedule
- Ideas for engagement questions
Focus on highlighting [SPECIFIC BENEFIT] and target [TARGET AUDIENCE].

Example Use Case: Creating a campaign to promote AI-assisted features.

General & Administrative

Policy Document Summarizer

markdown
Summarize the following [POLICY/DOCUMENT] into:
1. A one-paragraph executive summary
2. 5-7 bullet points of key takeaways
3. A list of any action items or compliance requirements
4. A simple table showing changes from previous version (if applicable)

Example Use Case: Distilling a lengthy updated security compliance document into an easily

digestible format for team distribution before quarterly compliance training.

Data Analysis Assistant

markdown
Help analyze this [DATA SET] to identify:
1. Key trends and patterns
2. Anomalies or outliers
3. Correlations between variables
4. Business implications
5. Suggested next steps or areas for deeper investigation

Provide both summary insights and specific data points that support your analysis.

Example Use Case: Business users analyzing quarterly expense reports to identify spending patterns across departments, flagging unusual transactions, and generating cost-saving recommendations for leadership.

Product Division

User Story Generator

markdown
Create detailed user stories for implementing a [FEATURE]. For each user story:
- Follow the format: "As a [USER TYPE], I want to [ACTION] so that [BENEFIT]"
- Include acceptance criteria
- Suggest story points (1, 2, 3, 5, 8)
- Identify potential dependencies
- Tag with appropriate labels (frontend, backend, UX, etc.)

Example Use Case: Developing comprehensive user stories.

Code Visualization Assistant

markdown
Visualize the following code to help understand:
1. Execution flow
2. Function calls and relationships
3. Key dependencies
4. Potential bottlenecks

Present the visualization in [FORMAT] (Mermaid, ASCII art, etc.)

Code:
[PASTE CODE HERE]

Example Use Case: Product managers and developers collaborating to better understand the

implementation of a complex feature by visualizing code execution paths and dependencies before planning refactoring work.

AI Workflow Prompts

Data Set Analysis

markdown
Analyze the attached data file and provide:
1. Summary of key content
2. Important patterns or trends
3. Insights grouped by priority
4. Recommendations based on findings

For [TYPE] data, focus on [SPECIFIC METRICS].

Example Use Case: Analyzing customer usage data to identify adoption patterns of features, highlighting which user segments are most actively engaged with specific tools.

Code Modernization Assistant

markdown
Review this legacy code and provide:
1. Explanation of current functionality
2. Modern implementation approach
3. Key improvements in the new version
4. Implementation considerations and challenges

Original code:
[PASTE CODE HERE]

Example Use Case: Modernizing legacy automation scripts to current best practices, improving maintainability while preserving essential functionality that teams depend on.

SECTION: tools-and-tips/ai/gitlab-duo.md

Learn how to use AI-powered GitLab Duo Chat, Code Suggestions and more to make your DevSecOps workflows faster.

Access

If you require access to GitLab Duo for the gitlab-com group please create a HelpLab ticket to work with IT to get GitLab Duo enabled for the group.

GitLab contributors and co-creators can also take advantage of AI-powered GitLab Duo. Start with the onboarding process in contributors.gitlab.com/.

If team members need access in customer demo group on GitLab.com, create an Access Request using the GitlabComLicensedDemoGroupRequest template.

Follow the Getting Started documentation to onboard.

GitLab Duo in IDEs

For IDE integration through GitLab Duo extensions, follow the editor extensions documentation.

Resources

- GitLab Duo documentation
  - Getting Started
  - Use cases
  - Duo Code Suggestions
  - Duo Chat
- GitLab University
  - AI and GitLab Duo courses
  - GitLab Duo Enterprise learning path
- Developer Advocacy resources
  - Content library with GitLab Duo demos, use cases, product tours, talks, workshops, recordings, etc.
- Highspot: Field guide (internal only, field teams)
- Dogfooding: Developing GitLab Duo blog tutorial series

Tips

GitLab Duo Chat can answer many questions about GitLab, programming languages, technology and more. Use it for your advantage, and practice how to ask questions and create follow-up conversations, instead of opening multiple browser search tabs. Explore, experiment, and iterate on chat prompts and responses. Learn more in the blog post 10 best practices for AI-powered GitLab Duo Chat.

If you are using GitLab Duo to write code, dive into the blog post Top tips for efficient AI-powered Code Suggestions with GitLab Duo.

Explore the use cases in this handbook page for GitLab team members and co-creators. More use cases and workflows are documented in the GitLab Duo documentation.

Open feature requests:

1. Ask GitLab handbook questions in GitLab Duo Chat

Handbook use cases

Preparation steps for handbook edits

Ensure that GitLab Duo Chat works in your IDE or GitLab UI.

1. GitLab UI

1. Web IDE
1. VS Code
1. JetBrains IDEs (IntelliJ IDEA, PyCharm, and others)

Keyboard shortcuts to access Duo Chat in VS Code/Web IDE:

1. Use cmd shift p (macOS) to open the command palette.
1. Search for GitLab Duo Chat and press Enter.
1. Optional: Move Duo Chat to the right panel by dragging it there explained in [this video].

If you want to use Code Suggestions to write and complete Markdown content, you need to configure markdown as additional language in the IDE extension settings. Tip: Multiple open tabs and more file content can help increase the context and quality of suggestions. Schedule a coffee chat with @dnsmichi, handbook backend maintainer, and ask for live screenshare learning sessions.

Create Markdown tables

Problem to solve: Is there a way to add a table on a handbook page via web IDE?

Use the following prompt to create a table with preseed data columns:

markdown
Create a Markdown table with the following data set
Header: Cloud, GPU type, Costs, Spec, Notes
Fill the entries with sample data for 3 rows.

GitLab Duo Chat may visualize the Markdown table. Use that to your advantage to verify that the result matches your expectations, and follow up with a prompt to define the output format as raw Markdown.

markdown
Show the raw Markdown in a code block

> Note: The same workflow is available in GitLab Duo Chat in local IDEs with the Duo extension.

Update or refactor Markdown tables

Sometimes, Markdown tables need to be split into multiple tables, or merged into a single one. Or an additional column needs to be added.

1. Open the IDE and select the table that should be updated or refactored.
1. Ask GitLab Duo Chat the following prompt:

   markdown
   /refactor the table for better readability. Split it by the first column into separate tables.

1. When Duo Chat visualizes the table in the response, use that to verify that the table is correctly divided. You can follow up with a prompt to ask for a raw output:

```
markdown
Only show the refactored table as raw Markdown code blocks
```

Development use cases

Read the blog post Developing GitLab Duo: How we are dogfooding our AI features to learn how to

1. Streamline the code review process
1. Condense comment threads
1. Create new documentation
1. Craft release notes
1. Optimize docs site navigation
1. and more

Troubleshoot failed CI/CD pipelines

1. Navigate into the failed pipeline's job view, and inspect the log. Follow the steps in the documentation to start the Root Cause Analysis.
1. Use the chat prompt to ask follow-up questions, for example how to prevent the error long term.

You can explore the use cases in:

1. Duo Enterprise product tours and Root Cause Analysis challenges maintained by the Developer Advocacy team
1. GitLab University: GitLab Duo Enterprise course

Onboarding and contributions

Team members and community contributors can take advantage of AI-powered workflows for fast onboarding, learning more about the code base and GitLab, and contributing with faster review cycles.

1. Learn about the source code base, and explore how to implement a specific feature proposal or bugfix.
1. Speed up Merge Request summary and code reviews.
1. Troubleshoot failing CI/CD pipelines.

Learn more in the GitLab Duo use cases documentation.

Terminal Integration with GitLab Duo Quick Chat

Problem to solve

Remembering command line arguments is hard and searching through man pages is time-consuming.

Solution

Integrate GitLab Duo Quick Chat with your terminal to help with command-line operations.

How to set it up

1. Configure your terminal to use VSCode as the default editor by adding this to your shell profile (.bashrc, .zshrc, etc.):

```
bash
export EDITOR="code --wait"
```

2. When you need help with a command:
   - Begin typing the command in your terminal (e.g., git rebase -i)
   - Press <kbd>Ctrl</kbd>+<kbd>x</kbd> <kbd>Ctrl</kbd>+<kbd>e</kbd> to open your current command line in VSCode

3. In VSCode:
   - The command you were typing appears in a new temporary file
   - Open GitLab Duo Chat (using <kbd>Cmd</kbd>+<kbd>Shift</kbd>+<kbd>P</kbd> on macOS or <kbd>Ctrl</kbd>+<kbd>Shift</kbd>+<kbd>P</kbd> on Windows/Linux and type "GitLab Duo Chat")
   - Select the command text (or delete it) and ask Duo to help you achieve your goal. For example: "Help me write a git command to squash my last 3 commits into one" or "I need a command to rebase my current branch onto main and resolve conflicts interactively"

4. Use Duo's response:
   - When Duo generates the improved command, click the Insert Snippet button in its response
   - Save the file (<kbd>Cmd</kbd>+<kbd>S</kbd> or <kbd>Ctrl</kbd>+<kbd>S</kbd>) and close the tab
   - The edited command will appear in your terminal, ready to execute

Additional notes

- This workflow also works with the integrated terminal in VSCode
- Keyboard shortcut <kbd>C-x</kbd> <kbd>C-e</kbd> is a standard bash/zsh feature for editing the current command line, not specific to Duo

---
SECTION: tools-and-tips/ai/security_division_ai_usecases.md

Learn how the Security Division leverages AI platforms such as Claude and GitLab Duo to optimize workflows, improve productivity and automate manual tasks.

Security Tools Using AI

The Security Division integrates AI capabilities into various tools and automations. Most use cases integrate AI capabilities into existing tools and capabilities to improve productivity and automate manual processes and tasks.

Tool	AI Engine	Use Case	Team
AI-assisted Incident reporting	Claude	Helps users to report security issues more quickly and efficiently by pre-filling the incident report forms based on a short description of the issue.	Security Incident Response Team (SIRT)
AppSec Assistant	GitLab Duo	AI reviews development issues to identify security risks at design time	Product Security Engineeringon behalf of Application Security
Automate our Continous Control Monitoring Program	Claude	Generate entire scripts used for: Pull data from resources (e.g. AWS); Pull policies from a yaml file; Perform an audit analysis and Conclusion (example here)	Security Compliance
Generation of Test Cases for gitlab-assistant	GitLab Duo	Generate basic and complex test cases for a Python module that standardizes scripting of solutions across the team when building automations and functionality for interactions with GitLab. The module introduces business logic beyond the basic API endpoint interactions.	Security Assurance
GitLab Duo generated CVE descriptions	GitLab Duo	Generates CVE description from the imported HackerOne reports that can optionally be used in our CVE's.	Application Security
Incident Summarization (/sirtsummary)	Claude	Incident summarization slash command in Slack	Security Incident Response Team (SIRT)
Sir Tanuki (SIRT Incident Review Bot)	GitLab Duo	The SecOps Incident Reviewer (Sir) Tanuki is SIRT·s GitLab Duo-powered incident report reviewing tool. It works by leveraging GitLab Duo to analyze security incident issues and give feedback on the different sections of the issue description.	Security Incident Response Team (SIRT)
TLDR Customer Threat Detections	Claude	Signals Engineering uses Claude to generate new TLDR threat detections - a Slack slash command (/tldr) is available to kick off a new Claude written MR for review.	Signals Engineering
Signals Engineering Auto Metric Stats	Claude	Every week before the weekly SET meeting, Claude auto-summarizes comments SIRT Engineers left when closing alerts during the past week which get added to a weekly detection digest issue	Signals Engineering

AI Driven Process Efficiencies

Process	AI Engine	Efficiency Details	Team
Tableau Data Manipulation	Claude	Leverage Claude to generate syntax for calculated fields in Tableau to enable data manipulation. These fields enable us manipulate existing data and create new dimensions and measures to support the Security metrics program.	Security Governance
Policy Generation and Optimization	Claude	Create the foundations of Security and Technology policies and reduce verbosity of policy language to align with Governance expectations.	Security Governance
Security Training Content Script Creation and Editing	Claude	Create scripts for AI created Security Training videos and editing of Security training content for readability and conciseness.	Security Governance
Blog and White Paper Optimization	Claude	Optimize the language and readability of blog posts and white papers to support a polished product for customers and the community.	Field Security

Ideas, Experiments and Tests

The security division works out of GitLab issues to keep track of AI-integrated ideas,

experiments, and tests. Generally it is a good idea to add the AI GitLab label to issues for tracking.

SECTION: tools-and-tips/editors-and-ides/_index.md

Overview

We encourage everyone to use the right tools for the job, and since
everyone here does a lot of writing, using the
correct editor for you is really important. Developers will have more specialized needs, but everyone at GitLab needs
to write Markdown, and the best text editors make this much easier.

A good editor/IDE will have:

- syntax highlighting, showing structure as well as content
- plugins to enable customization (such as spell-checking)
- integration with git, enabling managing commits and branches from within the editor
- the ability to open very large files efficiently
- not require a persistent internet connection
- powerful and fast search facilities
- the ability to automatically navigate between code declarations and usages in a large codebase

Many editors are free to use, some require paid licenses to unlock all features.
GitLab team members can expense the cost of their tools, and in some cases we
have licenses we can hand out (for example, for JetBrains IDEs).

There are many, many good text editors.

Sub-pages

See the following sub-pages for details on editors and IDEs used with GitLab

---
SECTION: tools-and-tips/editors-and-ides/emacs.md

Website: <https://www.gnu.org/software/emacs/>

Best for: code editing in the terminal

Strengths:

- Infinitely customizable
- Best in class plugins and extensions (many ideas start in emacs, and disseminate outwards)
- org mode
- Great git integration (magit)

- built-in email client

So powerful it is almost comparable to an operating system, emacs is an extremely powerful text editor that
can be customized in code.

SECTION: tools-and-tips/editors-and-ides/gitlab-web-ide.md

Website: <https://docs.gitlab.com/ee/user/project/webide/>

Best for: Editing directly from the GitLab web UI

Strengths:

- GitLab's official version of Visual Studio Code which can be run directly for any project or file within GitLab
- It also offers integration with GitLab's Workspaces feature.

SECTION: tools-and-tips/editors-and-ides/sublime-text.md

Website: <https://www.sublimetext.com/>

Best for: entry level developers, editing markdown.

Strengths:

- simple interface
- cross-platform
- many extensions, including one for integration with GitLab (GitlabIntegrate).
- powerful visual git-client integrated (sublime-merge)

A powerful text editor, sublime text is an excellent choice for editing plain text documents, but scales
up to full code-editing. Installing extensions is simple.

Configuration

Putting the following in Preferences.sublime-settings - User will among other things ensure that if you open the www-gitlab-com website you're not opening the output files by accident:

```
css
{
  "fontsize": 18,
  "spellcheck": true,
  "translatetabstospaces": true,
  "trimtrailingwhitespaceonsave": true,
```

```
  "folderexcludepatterns": ["public"]
}
```

SECTION: tools-and-tips/editors-and-ides/vim.md

Website: <https://www.vim.org/>

Best for: code editing in the terminal

Strengths:

- Extremely configurable
- Keyboard orientated
- Efficient modal editing
- Plugins for all code editing tasks
- Great git integration (fugitive)
- Spell-check built-in by default

An extremely powerful advanced editor, with a steep learning curve. While many users swear by this program,
it can be a challenge to get used to modal editing (or to unlearn it when you have).

SECTION: tools-and-tips/editors-and-ides/visual-studio-code.md

Website: <https://code.visualstudio.com/>

Best for: code editing, GitLab integration

Strengths:

- Extremely popular
- Vast number of extensions, including an official GitLab extension
- Language server protocol (LSP) was designed for vscode

A powerful editor, suitable for all kinds of editing, with a shallow learning curve. The LSP system was
designed for vscode, so vscode has excellent support for this transformational technology.

SECTION: tools-and-tips/editors-and-ides/jetbrains-ides/_index.md

Sub-pages

See the sub-pages for information on configuration and usage of
JetBrains IDEs in general, and for specific usage of IDEs.

Overview

JetBrains offers a suite of powerful integrated development environments(IDEs)
for all major software development ecosystems.

While they have a somewhat steep learning curve, JetBrains IDEs have many benefits which can make the investment worth it:

1. Common UI: Although they are separate applications, each IDE shares a common UI and controls, which allows you to
   easily switch between them without re-learning UX or keybindings.
1. Powerful proprietary features: The proprietary support for refactoring, indexing, searching, type checking, code navigation, etc., especially
   for languages without official language server support, is often much more powerful and faster than what is available
   from other editor ecosystems.
1. Works out of the box: Since they are language/ecosystem specific, many features work "out of the box", without the
   need to find, install, or configure any custom plugins or extensions. For example, ESLint and RuboCop have native
   support, with no plugins required. However, "power users" or complex projects
   will often want to customize their configurations.
1. Curated plugin ecosystem The JetBrains plugin ecosystem is (subjectively) more "curated" than other editor ecosystems.
   Most important tools which are not built into the IDE have officially supported plugins provided by JetBrains
   (e.g. VueJS, Prettier, NodeJS, etc.), and most popular non-JetBrains plugins only have one or a small number to choose from.
   This is in contrast to plugin ecosystems such as VS Codes, where there can be dozens of different plugins for each key tool
   or library, without a clear way to choose between them, and sometimes they will conflict with each other in keybindings or
   behavior.

JetBrains IDEs are widely used by many developers. The actual usage numbers are often hard to interpret,
because most surveys and polls compare each individual IDE (e.g. RubyMine vs. PyCharm vs. IDE) against
other non-specialized editors (e.g. vim, emacs, VS Code). But, based on recent surveys,
a rough estimate is that about 15% of professional software developers today use one or more JetBrains IDEs.

Setup and Configuration

See the Common JetBrains Setup and Configuration page for instructions on installing and configuring JetBrains IDEs.

Recommendation

Please do not use personal access tokens (PAT) on plug-ins for security reasons. For GitLab

Duo, please use an API token with scope aifeatures

Debugging

See the following IDE-specific pages for tips on debugging:

- RubyMine debugging

Keymaps

There's a lot of keybindings in JetBrains IDEs. Here's a list for RubyMine: <https://www.jetbrains.com/help/ruby/mastering-keyboard-shortcuts.html>. You can find the ones for their other IDEs too.

But if you only memorize one keyboard shortcut in JetBrains, make it this one:

- "Find Action": Cmd-Shift-A" (Ctrl-Shift-A` on Windows/Linux)

This will let you find a command and execute it, open a tool window, or search for a setting just by typing its name, and you can also see the menu location and shortcut (if defined) next to it.
It is similar to "Command Palette" (Cmd-Shift-P) in VS Code.

Although it is a good practice to learn the default keymaps, you will probably want to customize some of your keymaps. See the Configuration section
for details and examples on how to configure your own keymap additions/overrides, or copy someone
else's.

Code Inspections

One of the powerful and productivity-enhancing features of JetBrains IDEs is
Code Inspections

See more details at Code Inspection

Tracked JetBrains Issues

We keep a list of all JetBrains issues which are relevant to GitLab, and we want to follow/upvote
in hopes that they eventually get fixed.

See the list here: Tracked JetBrains Issues

Getting Help

If you need help, ask in one of the Chat Groups!

Chat Groups

- jetbrains-ide-users internal Slack channel for GitLab team members.
- ext-jetbrains-gitlab-support] shared Slack channel for JetBrains and [GitLab team members.
This is a private channel and you can request access in the jetbrains-ide-users channel.

Licenses

If you are a GitLab team member, see the Licenses page for details on how to get
JetBrains IDE licenses.

SECTION: tools-and-tips/editors-and-ides/jetbrains-ides/code-inspection/_index.md

Code Inspections

(If you are here to ask "What are all the noinspection comments in the code?", please read the "Why are there noinspection comments" page for a thorough explanation.
But if you just want to learn how to use Code Inspection effectively in JetBrains, this is the page you're looking for...)

One of the nice features of JetBrains IDEs is "Inspect Code" (Code -> Inspect Code).

See a description of this feature here: <https://www.jetbrains.com/help/ruby/running-inspections.html>

The sections below give more details on how to take full
advantage of this feature.

Keeping a clean "Inspect Code" and "green check"

This allows you to run all code inspections, including project-configured supported linters such as RuboCop/ESLint, as well as custom IDE inspections such as type safety, potential bugs, etc., which will show up in the "Problems" pane after you run "Inspect Code".

In addition to the "Problems" pane, you get instant feedback in the editor with the problems being underlined, as well as a summary of all problems with icons in the upper-right of each file.
All of these can be hovered over with the mouse to give you more options to deal with the errors,
including automatically fixing them in some cases.

And if you have no problems in the file, you get a "Green Check" at the top right of the file, as
a constant and instant feedback.

These can be powerful time-saving tools to catch problems immediately, without having to wait for
a pre-push hook like "LEFTHOOK" to catch them, or ever worse, having to wait for CI to catch them.

"Next Error" shortcut

PRO TIP: If you set
Settings -> Editor -> Code Editing -> Error Highlighting -> The 'Next Error' action goes through
to the "All Problems" selection, you can use F2 to cycle through all errors in the file,
then use Alt-Enter and arrow keys to show the options you have for resolving the error.

Suppressing false positives with noinspection comments

Sometimes, though, there are "false positive" problems reported by "Inspect Code". These are usually due to some IDE bug or limitation, or sometimes due to incorrect type annotations in a library.

Whatever the cause, it's good to suppress these false positives, because:

1. They cause noise in the "Problems" pane, and have to be ignored. Or in the case of a file with
   no other real problems, cause the "Problems" pane to pop up when it otherwise would not.
1. They prevent the nice "Green Check" from showing up in a file with no other real problems.
1. For JetBrains users who new to that area of the code, they don't have a way to know if they are
   a legit problem they should consider fixing or not.

The way you suppress these false positives is with noinspection ... comments:
<https://www.jetbrains.com/help/ruby/disabling-and-enabling-inspections.html>

If you use
F2 "Next Error" then right arrow, you can automatically add the correct noinspection ... comment to ignore the problem. However,
you will have to manually add a space after the to avoid a RuboCop
warning in the gitlab project.

Use the following process to automatically add a noinspection comment:

1. Either hit Option-Enter while on the warning line in the code (after pressing F2 to find it),
   or else in the "Inspect Code" Problems pane report right-click on the error.
1. In the menu that comes up, find the Suppress for statement option and click it.
1. Note that there is currently a bug that doesn't put a space after the comment in Ruby,
you'll have to add one to avoid a rubocop error (TODO: Open an issue for this against RubyMine)
1. This enables you to have a nice, clean report and green check with no false positives :)

Each noinspection should be accompanied by an explanation or link

These noinspection comments might confuse some non-JetBrains users.

Therefore, each of these comments should be accompanied by a comment explaining why it was needed, with a reference to any supporting information.

If it was needed due to a JetBrains bug or limitation, you can reference the specific entry on the
Tracked JetBrains Issues page. If one doesn't exist yet, you should:

1. Open an issue against JetBrains and add it to that page (this is preferred).
1. Or, if you don't have time to identify or open an issue against JetBrains, leave a TODO: link pointing to that page, and
   indicating that a JetBrains issue should be identified or opened at a later time.

Each noinspection and explanation should be a one-liner

All noinspection - <some link or explanation> comments should be on a single line.

This is because we want to avoid the possibility of deleting an obsolete/fixed noinspection line
but forgetting to delete the separate associated comment line.

This should be done consistently, even if there needs to be an associated rubocop:disable Layout/LineLength,
on a separate line (because that will cause a separate RuboCop warning if it is left in while no longer
necessary).

Other options considered:

- We could use a URL shortener service, but that makes people concerned about the security of clicking
on links with an unknown destination.
  We did discuss and reject in the past the possibility of an internal URL shortener.
- We could leave some of the comment without an explanation of link, but that leaves us with the original
  problem of people not understanding their purpose, and having to manually address repeated questions
  around them, rather than people discovering the answer themselves in this section.

Using Inspect Code with custom scopes

One way you can make Code Inspections (and other JetBrains operations) faster and more powerful is through
the use of custom scopes

If you curate a custom "Scope" which only selects the files related to the feature you are currently working on, you can
also use this report to find all warnings/errors in any of those files.

Here's a quick list of steps to set this up (TODO: add more details/links):

1. Ensure you have all the linting plugins enabled and configured: rubocop, eslint, prettier (search for these in the Preferences search, some are enabled by default)
1. In Preferences -> Editor -> Code Editing -> Error Highlighting section -> Change The 'Next

Error' action goes through to All problems
1. Then you can see all the highlighting in the current file by default.
1. Press F2 (next error) to cycle through all errors in the current file. Press Option+Return while on the error to open a menu of possible fixes.
1. To check multiple files, you can use the Inspect Code function (Cmd-shift-A -> "Inspect Code")
   and pick an individual file or custom scope to inspect.
1. You can also set up a custom scope to only include the files for the Feature or area of code you are working on
    1. Preferences -> Appearance and Behavior -> Scopes
    1. + to add a scope and give it a name (e.g. remotedev)
    1. Use the Include/Exclude/Recursively buttons to define what files should be included in the scope.
    1. Here's a current example of the remotedev scope definition which could be shared with other team members: file[gitlab]:ee/lib/remotedevelopment//||file[gitlab]:ee/spec/factories/remotedevelopment//||file[gitlab]:ee/spec/lib/remotedevelopment//||file[gitlab]:ee/app/services/remotedevelopment//||file[gitlab]:app/models/remotedevelopment//||file[gitlab]:ee/app/graphql/mutations/remotedevelopment//||file[gitlab]:ee/app/graphql/resolvers/remotedevelopment//||file[gitlab]:ee/app/graphql/types/remotedevelopment//||file[gitlab]:ee/app/models/remotedevelopment//||file[gitlab]:ee/spec/graphql/types/remotedevelopment//||file[gitlab]:ee/spec/models/remotedevelopment//||file[gitlab]:ee/spec/services/remotedevelopment//||file[gitlab]:ee/app/finders/remotedevelopment//||file[gitlab]:ee/spec/features/remotedevelopment//||file[gitlab]:ee/spec/support/sharedcontexts/remotedevelopment//||file[gitlab]:ee/app/graphql/ee/types/userinterface.rb||file[gitlab]:ee/app/graphql/resolvers/concerns/remotedevelopment//||file[gitlab]:ee/app/graphql/resolvers/projects/workspacesresolver.rb||file[gitlab]:ee/app/graphql/resolvers/users/workspacesresolver.rb||file[gitlab]:ee/spec/requests/api/graphql/mutations/remotedevelopment//||file[gitlab]:ee/spec/requests/api/graphql/remotedevelopment//||file[gitlab]:ee/spec/finders/remotedevelopment//||file[gitlab]:ee/app/assets/javascripts/remotedevelopment//||file[gitlab]:ee/spec/frontend/remotedevelopment//||file[gitlab]:ee/spec/graphql/api/workspacespec.rb
    1. You can also share the XML file for the directly, it will be under .idea/scopes/SCOPENAME.xml.
    1. Here's Chad's current examples of two scopes related to Workspaces:
        1. All remote dev files: <https://gitlab.com/jetbrains-ide-config/jetbrains-ide-config-gitlab/-/blob/master/dotidea/scopes/remotedev.xml>
        1. Only remote dev services and lib files: <https://gitlab.com/jetbrains-ide-config/jetbrains-ide-config-gitlab/-/blob/master/dotidea/scopes/remotedevserviceslib.xml>

SECTION: tools-and-tips/editors-and-ides/jetbrains-ides/code-inspection/why-are-there-noinspection-comments.md

What are all the noinspection comments in the code?

(If you were given this referred to this page in response to asking that question, please read this page for a thorough explanation.
But if you just want to learn how to use Code Inspection effectively in JetBrains, please see the Code Inspection main page)

JetBrains is widely used

JetBrains is a powerful IDE, and is by far the industry leader in traditional "IDEs" (as opposed to all-purpose editors), with various surveys showing a significant percentage of developers using JetBrains IDEs (it is hard to get specific numbers, because most surveys split them into separate IDEs per ecosystem/platform/language such as RubyMine, Webstorm, etc., but compare these to multi-ecosystem editors such as VSCode, Vim, etc.).

Here's some various surveys showing usage of JetBrains editors:

- StackOverflow 2023 Developer Survey
- Ruby on Rails 2022 Community Survey
- JetBrains 2022 Developer Ecosystem Survey
- GitLab 2023 IDE/Editor usage (INTERNAL DOCUMENT LINK)

JetBrains supports powerful Code Inspection above and beyond what is provided by git hooks or CI

JetBrains IDEs include a powerful Code Inspection feature with inspections built into JetBrains IDEs, including type safety and other checks which are not provided by our standard static analysis tools such as RuboCop or ESLint.

This support can be a tremendous productivity boost for JetBrains IDE users, because you can get instant feedback on linting/type/etc. errors, without having to wait for LEFTHOOK pre-push hooks, or a CI pipeline to run and fail (a much longer and more tedious feedback loop).

This feature makes GitLab team members and contributors who use it more efficient

There are multiple JetBrains IDE users who are GitLab team members, and there is an active internal jetbrains-ide-users Slack channel.

Therefore, on some teams with several JetBrains users, such as the Workspaces team, we invest ongoing effort in keeping the Code Inspection for the feature clean and without any warnings/errors,
which means the "green check" at the top right of each file is useful, and if it's not there, we know
immediately that we have introduced some problem.

This approach also supports our mission of "Everyone can Contribute", because if a JetBrains external
contributor wants to contribute to an area of the code with curated noinspection comments, they can
do so without having to deal with any false positive warnings in the IDE, and trust that warnings they
see are likely due to problems which they themselves have introduced in their current coding session or MR.

We must suppress false positives to make full use of this feature

However, sometimes they give false positives, and must be disabled with noinspection comments to
keep things clean, similar to disable comments in RuboCop, ESLint, or similar static analysis

tools.

These false positives often indicate a bug or missing functionality in the IDE. Therefore, in these
cases, we will proactively report these to JetBrains, and track the corresponding issues in their issue tracker.

This tracking exists under Tracked JetBrains Issues, and the related
comments should all include the relevant issue entry as a reference. Once the underlying issues are
resolved and included in a new IDE release,
the corresponding noinspection comments can be removed.

But other noinspection comments are due to default JetBrains inspection rules we don't want to enforce because we are intentionally making an exception to the default rule. An example of this is
warnings about class/variable/module/method/constant names being too long or too short.
One option would be to just change the settings for the inspection, for example to allow longer names. However, this would require that everyone have these same overrides in their JetBrains configuration, which we do not want to require or rely on, especially for external contributors.

Not everyone has to use them, but please don't try to prevent others from using them

We also do not require that non-JetBrains users maintain these comments - the JetBrains users on the
team will take responsibility for maintaining them, adding explanatory comments/TODOs, tracking any
associated JetBrains issues, and removing fixed or obsolete ones.

We do request is that there are no
requests from non-JetBrains users to remove these comments,
unless the JetBrains Issue tracking the comment has already been resolved.

While it may not be useful for folks not using JetBrains IDEs, it does provide benefits to JetBrains users to write
quality code by ensuring no such warnings are present, and it is in support of our values of Efficiency, Results,
and Diversity/Inclusion/Belonging.

If you are using a non-JetBrains IDE which has significant usage at GitLab, and it has similar support in that IDE, we encourage
you to create such documentation on its benefits and how they should be used and how will they be tracked.

Concerns about proliferation of such comments in our codebase for different editors are well-founded, and we take them seriously.
Thus, team members are encouraged to open issues inviting discussion to remove comments from an IDE which is not being actively used at GitLab, or for
JetBrains noinspection comments in areas of the code which are not being

actively maintained/curated.

In reality, these comments are currently restricted in scope in the gitlab codebase. Currently, as of
this MR in July 2023, all noinspection
comments have been removed from the code except for within the Workspaces domain, which is the only group actively using it.

But now that we have standardized this process and added
handbook support for configuring and using JetBrains IDEs
, other team members and teams who are JetBrains users may also want to adopt this approach.

SECTION: tools-and-tips/editors-and-ides/jetbrains-ides/individual-ides/_index.md

Individual IDEs

JetBrains offers a suite of powerful integrated development environments(IDEs)
for all major software development platforms/ecosystems.

Unlike other single-application editors, each supported platform/ecosystem has a separate individual IDE application.

Here are the individual IDEs most often used at GitLab:

SECTION: tools-and-tips/editors-and-ides/jetbrains-ides/individual-ides/goland.md

Overview

Website: <https://www.jetbrains.com/go/>

Best for: Applications written primarily in Go.

Common JetBrains Setup and Configuration

JetBrains IDEs are standardized, so much of the setup and configuration information applies to all IDEs, and can be found under Common JetBrains Setup and Configuration.

Specific config for GoLand can be found in the sections below.

GoLand-specific Setup and Configuration

Placeholder for future content...

SECTION: tools-and-tips/editors-and-ides/jetbrains-ides/individual-ides/rubymine.md

Overview

Website: <https://www.jetbrains.com/ruby/>

Best for: editing Ruby or Rails applications, which can include Javascript/Typescript and most other
web technologies.

Common JetBrains Setup and Configuration

JetBrains IDEs are standardized, so much of the setup and configuration information applies to all IDEs, and can be found under Common JetBrains Setup and Configuration.

Specific config for RubyMine can be found in the sections below.

Using Rubymine debugger for GitLab running under GDK

Debugging rails-web

Note on Rails Puma binding

By default, the Rails puma configuration template which GitLab uses binds to a socket, instead of a TCP port.

This MR added support for configuring the GDK to bind Rails puma to a TCP socket, and added support for Workhorse to use TCP ports instead of sockets, and this JetBrains issue added support for starting a socket-based Puma server in a Run Configuration.

See the architecture documentation around components for more details on how GitLab Workhorse and Puma work together.

Setting up a RubyMine "Ruby" Run Configuration with puma using default socket binding

1. Make sure you have done gdk stop rails-web before each debugging session (and gdk start rails-web when you are done debugging)
1. In RubyMine, open the gitlab repo directory from /path/to/gdk/gitlab (Cloning gitlab and running it from a different directory won't work out of the box)
1. Go to Run -> Edit Configurations to set up a Rails Run/Debug config like this in RubyMine:
   - Name: GitLab: rails-web
   - Configuration:
     - Server: Puma
     - IP address: BLANK (delete 0.0.0.0)
     - Port: BLANK (delete 3000)
     - Environment: development
     - Environment Variables (Note: these are taken from the current GDK Procfile, as well as additional ones to prevent timeouts during debugging. They may become outdated):
       - RAILSRELATIVEURLROOT=/;ACTIONCABLEINAPP=true;ACTIONCABLEWORKERPOOLSIZE=4;P
se;GEOSECONDARYPROXY=0;GITLABRAILSRACKTIMEOUT=999999;GITLABRAILSWAITTIMEOUT=99
EREQUESTLIMITS=false;PUMAWORKERTIMEOUT=99999999
       - NOTE: The following values from the Procfile entry are omitted as they are not

necessary:
    - BUNDLEGEMFILE
    - ENABLEBOOTSNAP
    - RAILSENV
1. Start the config in Run ("Play" ·· button) or Debug mode (at top right)

Debugging rails-background-jobs

To debug services run as background jobs, you will need to set up debugging for rails-background-jobs, in addition to the rails-web debugger. The setup is similar, although you're connecting to the sidekiq process instead of puma.

1. Make sure you have done gdk stop rails-background-jobs before each debugging session (and gdk start rails-background-jobs when you are done debugging)
1. Go to Run -> Edit Configurations to set up a Ruby (NOT Rails) Run/Debug config like this in RubyMine:
  - Name: GitLab: rails-background-jobs
  - Ruby Script: /Users/YOURUSER/.asdf/installs/ruby/RUBYVERSION/bin/sidekiq
  - Working Directory /Users/YOURUSER/PATHTO/gitlab-development-kit/gitlab/
  - Environment Variables (Note: these are taken from the current GDK Procfile, they may become outdated): SIDEKIQVERBOSE=false;SIDEKIQQUEUES=default,mailers,emailreceiver,hashedstorage:hashedstoragemigrator,hashedstorage:hashedstorageprojectmigrate,hashedstorage:hashedstorageprojectrollback,hashedstorage:hashedstoragerollbacker,projectimportschedule,servicedeskemailreceiver;SIDEKIQWORKERS=1;RAILSRELATIVEURLROOT=/;GITALYDISABLEREQUESTLIMITS=false
  - NOTE: The following values from the Procfile entry are omitted as they are not necessary:
    - BUNDLEGEMFILE
    - ENABLEBOOTSNAP
    - RAILSENV

Configuring GDK database connection

First, follow the "Access the database with a GUI"
instructions to reconfigure postgresql under the GDK to run on localhost.

Then configure the develoment database:

1. Go to File -> New -> Data Source -> Postgresql (or from the Databases tab on the right border)
1. Enter a name: gitlabhqdevelopment@localhost
1. Host: localhost
1. Port: 5432
1. Authentication User & Password
1. User - leave blank
1. Password: leave default, shows <hidden>
1. URL (auto-calculated from the above fields): jdbc:postgresql://localhost:5432/gitlabhqdevelopment
1. Download driver if needed.
1. Click Test Connection.

Then access the database:

1. Open the Databases panel from the right border
1. Click the Refresh button (circle arrows)
1. Expand to see tables/views/etc.

If you want to add more schemas from config/database.yml:

1. Go to Database tab
1. Right click on top level of database, and view Properties (or the "wrench" button)
1. Go to the Schemas tab
1. Select the schemas you want.

Miscellaneous

- To enable "Navigate to Test" (<kbd>Cmd</kbd> + <kbd>Shift</kbd> + <kbd>t</kbd>) for EE code, right click on ee/spec directory and choose "Mark Directory As...", "Test Sources".

SECTION: tools-and-tips/editors-and-ides/jetbrains-ides/individual-ides/webstorm.md

Overview

Website: <https://www.jetbrains.com/webstorm/>

Best for: Javascript/Typescript and node.js applications and other web technologies, which do not
have a backend based on Ruby/Rails or other non-JS/non-Typescript languages.

Common JetBrains Setup and Configuration

JetBrains IDEs are standardized, so much of the setup and configuration information applies to all IDEs, and can be found under Common JetBrains Setup and Configuration.

Specific config for WebStorm can be found in the sections below.

WebStorm-specific Setup and Configuration

Placeholder for future content...

SECTION: tools-and-tips/editors-and-ides/jetbrains-ides/licenses/_index.md

Licenses

For GitLab employees, there is a central license management for JetBrains' products.

1. Depending on the Product you want to use, you might want or need to file an Access Request

1. If you want to onboard a larger set of users (5 or more), please do file an Access Request.
1. If you want to use any of the commonly used JetBrains products (listed below) you do not need to file an Access Request:
    - GoLand, RubyMine, PyCharm, DataGrip or WebStorm
    - IntelliJ IDEA Ultimate if working on Editor Extensions
1. If you want to use any other JetBrains product:
    - on a regular basis, please do file an Access Request.
    - occasional use is possible without an Access Request. Please note that we have a limited set of "All Products" licenses. People who filed an Access Request will receive priority treatment over occasional users.
1. You can activate a license for yourself even while an Access Request is pending.
    - Use the license server URL https://gitlab.fls.jetbrains.com during activation
    - Make sure you log in via Okta to authenticate

If you have an Individual License acquired through your own means, it is suitable for General commercial use, and you may use it. However, GitLab will not reimburse an Individual License, as Individual License cannot be purchased or reimbursed by companies. That being said, even if you used to use an Individual License, you can always request a company issued one.

SECTION: tools-and-tips/editors-and-ides/jetbrains-ides/setup-and-config/_index.md

Overview

JetBrains IDEs are standardized, so much of the setup and configuration information applies to all IDEs, and is consolidated on this page.

Any setup and configuration and other info that is specific to individual IDEs will be found on these sub-page under Individual IDEs. If this is the case, a link will be provided to the relevant section in the specific IDE's sub-page.

Getting Help

1. If you need help, ask in one of the Chat Groups!

Install the IDE

1. Decide what IDE you need to use for the project you are working on:
    - RubyMine for Ruby/Rails/JS
    - GoLand for golang
    - Webstorm for pure-Javascript/Typescript.
    - Idea for JVM/Java/Kotlin
    - CLion for Rust
1. If you are an GitLab employee, you can request and obtain a License for the IDE(s) you need to use.
1. Install JetBrains Toolbox.
1. Use JetBrains Toolbox to install the IDE you need.

At this point, you should be able to run the IDE and open the project you need to work on. For basic tasks, most things "just work" by default out of the box - that's one of the nice things about using JetBrains IDEs!

However, note that very large and complex projects such as the GitLab Rails monolith under RubyMine may an exception to this. Continue on with the rest of the instructions on this page for more details.

Indexes

The most important thing to do related to indexing is to set up the proper excluded folders. See the
Set up excluded folders section below.

Indexing Overview

One of the things that makes JetBrains IDEs powerful is their proprietary indexing technology, which indexes all of your project and dependency code, and
allows for powerful and near-instantaneous searches and navigation across the project.

However, when you first open a new project for the first time, or if you pull in significant updates to a project, it can take a while to update the indexes,
especially on a slower machine.

And for very large projects, such as the GitLab Rails monolith, which has
more than 2 million lines of Ruby code, and who knows how many millions of lines of gem code dependency, the very first indexing can take a very long time, up to 15-20 minutes or more depending on your machine. However, after that initial indexing, subsequent incremental indexing after pulling new changes or switching branches usually takes no more than a minute or so.

As long as you've already set up the proper excluded folders,
the best advice here is to just "be patient" - the good news is that it is only required once, when you first open the project or pull in new changes. Use this opportunity to take a little break, do a little stretching... ··

Shared Indexes

JetBrains does offer support for Shared Indexes for RubyMine (and other IDEs), which have the potential to greatly reduce the indexing time for people setting up the GitLab project for the first time.

However, this seems to be an involved setup process. If you are interested in helping set up and maintain shared indexes, please let us know in one of the
Chat Groups!

Set up excluded folders

It's important to set up the excluded folders, otherwise indexing and searching will take much longer

than necessary.

This will be under Settings -> Project Structure in RubyMine and GoLand, and under Settings -> Directories under WebStorm.

If you want to see a current example of the RubyMine excluded folders for the gitlab project, you can see Chad's gitlab.iml module at
<https://gitlab.com/jetbrains-ide-config/jetbrains-ide-config-gitlab/-/blob/master/dotidea/gitlab.iml>,
and search for excludeFolder.

Alternately, these would be included in the config if you use one of the "copy someone else's config"
Configuration approaches under Configuration.

Set up test roots

You should properly set up the test sources root directories.

Ensure that you add both spec and ee/spec as test roots.

This has several effects, including making that Cmd-Shift-T "Go to Test Subject" action work to easily toggle back and forth between the subject file and test file.

You can review existing test roots under Settings -> Project Structure.

Increase maximum heap size in memory settings

When working with the GitLab project, which is BIG, RubyMine can use a lot of memory when indexing/searching/etc. It's not unusual for memory usage to peak out at up to 10 gigabytes or more of memory during indexing, if it is allocated.

If you have the memory to spare on your workstation, it will help your performace to allocate a lot of memory. 12 gigabytes is a good start.

1. Help menu -> Change Memory Settings
1. Change Maximum heap size to 12000 Mib, or whatever you think you can allocate without otherwise impacting system performance. On a maxed-out MacBook pro with 64G of memory, allocating 12G should be fine.

Open files in RubyMine from Terminal

This can be set up at the OS level so it works for all type of files. For example, to set up open all .rb files:
1, Open a .rb file in Finder
2. Right click and select Get Info
3, Expand Open With
4. Select RubyMine.app
5. Select Change All...

Configuration

UPDATE 2025-05

Toolbox Enterprise has been renamed to IDE Provisioner in the IDE Services suite:

> Propagate global IDE settings
>
> With IDE Provisioner, you can define and propagate global IDE settings to all instances of the IDEs running in your organization. Set custom VM options, limit maximum heap size, define default code styles, and manage other properties on a per-profile, per-team, or company-wide basis.

UPDATE 2024-04

JetBrains has pointed us to this issue to follow their progress on allowing team settings sharing: Make Settings Sync/new separate feature suitable for team settings sharing

UPDATE 2023-12

Based on demo previews, it looks like the new Toolbox Enterprise features will finally provide a viable and easy way to share configuration across a team. JetBrains has said that the features we need should be available sometime around mid-2024. However, the additional license cost of this may be prohibitive.

UPDATE 2023-11

With the deprecation of the Settings Repository feature, JetBrains has acknowledged that there is currently no supported, non-deprecated way to share settings among team mambers. This is the current description of that issue:

> The new Settings Sync works with the JetBrains account; it makes this solution inappropriate for team settings sharing when it's necessary to share only certain settings (code style, color schemes, etc) between team members (they have their own JB Accounts).
>
> We already have the Settings Repository plugin, but it can be used only with third-party sources, and we don't maintain this plugin for now.
> The import/export settings feature is not automated enough.
> This feature may be implemented along with making profiles for Settings Sync.

On the jetbrains-ide-users internal Slack channel for GitLab team members, JetBrains has indicated that:
"one of the possible solutions should be implemented in the scope of Toolbox Enterprise, but AFAIK, there is no ETA for now"

However, if you are only wanting to sync your own settings to a remote git repo and not necessarily share them, that's still possible.

Since the deprecation of the Settings Repository plugin (and the fact that it seems to have some bugs they aren't planning to fix),

the Settings Sync feature seems to be the choice to do this,
which is listed below as "JetBrains-supported option 1: Settings Sync"

TODO: So, all of the sections below could use a reorg and rewrite in light of this new information.

TL;DR

If you want to get your JetBrains (RubyMine) configured to work (and specifically work with GitLab):

1. Your best bet is probably "Manual Option 1: Manually configure everything".
   This will take maybe an hour or so, but you'll learn your way around the settings.
1. If you are in a hurry, consider "Manual option 2: Manually zip and copy IDE and Project config folders" and ask on Chat Groups for a zip of someone's config folders
1. If you just want to backup or transfer your existing config, consider "JetBrains-supported option 1: Settings Sync"
1. If you have aspirations to be a JetBrains power user, or are the type of person who wants to keep all your configs under source control, see "JetBrains-supported option 2: Settings Repository" and "Manual option 3: Keep your .idea folder under source control".

Overview

NOTE: This Configuration section is a work in progress, especially since the options for managing/sharing configs have changed in recent versions of JetBrains. Please feel free to contribute any updates you may have.

Although most basic functionality works out of the box in JetBrains IDEs, if you work on a project with very specific and strict rules around code style and formatting (such as GitLab) you may need to carefully curate your IDE configuration to match the requirements of the project. For example, matching all code formatting and linting rules defined by the project.

Ideally, you would just want to import a pre-existing configuration that someone else is already curating and maintaining to keep it up-to-date with the project.

Unfortunately, this isn't as easy as it should be, and this is one of the areas that JetBrains IDEs could use a lot of improvement, as is evidenced by the 5 different approaches documented below.

This section will attempt to provide some solutions for this. But if you are new to JetBrains, please feel free to ask for guidance and advice in the Chat Groups.

"IDE" vs. "Project" config

JetBrains stores config in two ways: "Stored in IDE" and "Stored in Project":

- The "IDE" settings are stored under your home directory, in a directory like /Users/cwoolley/Library/Application Support/JetBrains/RubyMine2023.2

- The "Project" settings are stored in the .idea folder in the root of your project.

And some types of config allow you to choose which of those to use. It's probably better to store everything possible in the project.

Here's how you set that up:

1. Settings -> Editor -> Code Style -> Scheme -> "Project"
1. Settings -> Editor -> Inspections -> Scheme -> "Project Default"
1. Settings -> Editor -> File and Code Templates -> "Project"

Manual option 1: Manually configure everything

You can manually go into settings and make all the changes yourself.

Chad Woolley maintains documentation on how to manually configure JetBrains IDEs to work in an optimal and compatible way, specifically the GitLab monorepo.

If you have some time, this can be the simplest approach, and also lets you have more control than the other approaches of copying entire configs.

If you want to do this, you can just follow these instructions - it should take maybe a half hour to an hour.

- Chad Woolley's JetBrains IDE setup notes
- Chad Woolley's curated list of JetBrains overridden settings

These are a bit scattered as they have evolved over multiple years, projects, and IDE versions. Chad hopes to migrate and consolidate all these instructions to this page and sub-pages page real soon…

Manual option 2: Manually zip and copy IDE and Project config folders

Ask someone who already has the project configured to give you zipped copies of their IDE and Project config folders listed above. This SHOULDN'T contain any personal information other than perhaps dictionary override entries, but some plugins might store personal info in their own configs.

You should also ensure you are both on the exact same version of the IDE product.

Then you can backup your own folders (just in case), close the IDE, and unzip theirs to the same place. This should theoretically give you all their settings.

Also, Chad Woolley keeps his current .idea Project Config folder for the gitlab project committed and updated in source control (see instructions on how you can do this in the next section).

You can clone it or copy any files from this config here: <https://gitlab.com/jetbrains-ide-config/jetbrains-ide-config-gitlab/-/tree/master/dotidea>

Manual option 3: Keep your .idea folder under source control

If you want to keep your Project (not IDE) config under source control, you can do this manually. See Chad's instructions here:

<https://gitlab.com/jetbrains-ide-config/jetbrains-ide-config-gitlab/-/blob/master/README.mdproject-level-config>

JetBrains-supported options

JetBrains provides three different options for sharing settings, which are described on the following page (using RubyMine as an example):

<https://www.jetbrains.com/help/ruby/sharing-your-ide-settings.html>

1. Settings Sync
1. Settings Repository (DEPRECATED)
1. Settings Export/Import

Each of these have their tradeoffs, see the following sections for details.

JetBrains-supported option 1: Settings Sync

NOTE: With the deprecation of Settings Repository, this is the only option supported by JetBrains which allows you to store your settings in a git repo (see UPDATE 2023-11 note above).

This approach is described here: <https://www.jetbrains.com/help/ruby/sharing-your-ide-settings.htmlIDEsettingssync>

~~The main downside of this approach is that since this is done through your JetBrains account, there appears to be no way to share with other team members. So, you'll have to one of the other
approaches if you want to share your settings with someone else, or have them share theirs with you.~~

Update: As of RubyMine 2022.3, this (like the Settings Repository) also stores the config in a local git repo on disk
in the IDE Configuration Directory, which on MacOS is at:
/Users/cwoolley/Library/Application Support/JetBrains/<product><version>/settingsSync.

You can also manually manage pushing the repo to a remote, if you want to back it up outside your machine,
or let other team members view/copy individual config files.

Note that the downside of this is that is still only manages IDE-level config, not Project-level
config in the .idea folder of the project. If you want to back up your .idea folder too, you'll have to
manage that manually as described above.

JetBrains-supported option 2: Settings Repository

NOTE: This option is now deprecated and unsupported by JetBrains (see UPDATE 2023-11 note above).
You should use the Settings Sync option above as it is supported.

This involves syncing your IDE-level (not-project level) config to a git repo.

This approach has a few downsides:

1. It only manages IDE-level config, not project-level config
1. It's complex to set up, involving setting up a custom repo with HTTPS credentials
1. It doesn't support all git features (e.g. .gitignore seems to be ignored)
1. It doesn't seem well-supported by JetBrains, especially since they moved it out of being bundled with the IDE and made it a plugin.

The upside of it is that you can keep your IDE-level config under source control in your own repo, and not in JetBrains cloud as is done with "Settings Sync" above. This also means you can share it yourself as needed.

H